



CSCV2025 セキュリティ評価 およびプラットフォーム刷新プロジェクト

FPT Software 提案書

2025年5月

目次（1/2）

1. エグゼクティブサマリー

プロジェクトの背景、課題認識、推奨アプローチの概要

2. 現状分析

既存アーキテクチャの分析とCTFコンテキストの評価

3. セキュリティ評価

脆弱性分析、脅威モデル、リスク評価

4. 提案ソリューション

刷新アーキテクチャ、セキュリティ強化設計

5. 技術的アプローチ

実装戦略、DevSecOps、移行計画

目次 (2/2)

6. デモ計画

デモスコープ、ユーザージャーニー、成功基準

7. プロジェクト計画

タイムライン、体制、マイルストーン

8. 品質・コンプライアンス

QA体制、コンプライアンス対応、SLA

9. 費用内訳

コスト見積もり、ROI分析、比較検討

10. FPT Softwareの強み

選定理由、実績、差別化要素

Chapter 1

エグゼクティブサマリー

プロジェクトの背景

CSCV2025 (Cyber Security Capture-the-Flag 2025) は、セキュリティ評価とプラットフォーム刷新を目的としたプロジェクトです。既存の二層アーキテクチャ (Django APIゲートウェイ + PHPストレージサービス) を評価・強化し、セキュアなデモプラットフォームとして再構築します。

セキュリティ評価

既存システムの脆弱性診断とCTFベースの評価を実施

プラットフォーム刷新

評価結果に基づき、アーキテクチャの再設計とセキュリティ強化

デモ環境構築

刷新後のシステムをデモ環境として動作確認・提示

主要課題

ファイルアップロード脆弱性

PHPストレージサービスのファイルアップロード機能は、入力検証不足による任意コード実行の可能性を有します。

7zipアーカイブ処理リスク

gemorroj/archive7zライブラリを使用したアーカイブ展開は、Zip Slip攻撃などのパストラバーサル脆弱性を含みます。

認証・認可の未整備

JWT認証は実装されていますが、トークン管理、リフレッシュ、権限レベルの分離に改善余地があります。

コンテナセキュリティ

Docker Compose環境におけるネットワーク分離、権限制限、イメージスキャンが不十分です。

CTFフラグ: 既存システムには CSCV2025{for_testing} のテストフラグが含まれており、セキュリティ評価の起点となります。

FPT Softwareの推奨アプローチ

現状分析



脆弱性評価



再設計



実装・検証



デモ提示

ゼロトラスト設計

全ての通信を暗号化し、最小権限の原則でアクセス制御を実装

DevSecOps統合

CI/CDパイプラインへのセキュリティチェック統合、自動スキャン

継続的監視

リアルタイムのログ監視、異常検知、インシデントレスポンス体制

Chapter 2

現状分析

既存アーキテクチャ



コンポーネント	技術スタック	役割	ポート
App1	Python / Django / uWSGI	REST APIゲートウェイ、JWT認証	8000
App2	PHP / Apache	ファイルストレージ、7zipアーカイブ展開	80
Orchestration	Docker Compose	コンテナオーケストレーション	—

注意: App1は STORAGE_URL=http://app2 でApp2へHTTP（暗号化なし）で接続しています。

App1: Django APIゲートウェイ 詳細

技術構成

- Django REST Framework
- JWT認証 (SimpleJWT)
- uWSGIサーバー
- Dockerコンテナ化

主要機能

- ユーザー認証・認可
- ファイルアップロードAPI
- ストレージサービス連携
- ヘルスチェックエンドポイント

既知の懸念事項

- 内部通信の暗号化なし
- リクエスト率制限の未確認
- CORS設定の確認が必要

改善候補

- mTLSによるサービス間通信
- レートリミッティングの実装
- 詳細なログ記録と監視

App2: PHPストレージサービス 詳細

技術構成

- PHP + Apache
- gemorroj/archive7zライブラリ
- ファイルシステムストレージ
- Dockerコンテナ化

主要機能

- ファイルアップロード処理
- 7zip/ZIP/RARアーカイブ展開
- フラグファイルのマウント
- ヘルスチェック（未実装）

高リスク項目

- ファイル名検証の不備
- パストラバーサル可能性
- アーカイブ内の悪意あるファイル

改善候補

- ファイルタイプ検証の強化
- サンドボックス化された展開
- コンテンツセキュリティポリシー

Chapter 3

セキュリティ評価

脆弱性評価サマリー

ID	脆弱性	CVE	重大度	影響
VULN-01	ファイルアップロードによる任意コード実行	CWE-94	Critical	完全なシステム乗っ取り
VULN-02	Zip Slip — パス TRAVERSAL	CWE-22	Critical	ファイルシステム書き込み
VULN-03	サービス間通信の平文送信	CWE-319	High	中間者攻撃
VULN-04	JWTトークン管理の不備	CWE-613	High	認証バイパス
VULN-05	コンテナ権限の過剰付与	CWE-250	High	コンテナエスケープ
VULN-06	インメモリフラグファイルの露出	CWE-538	Medium	機密情報漏洩

脅威モデル — STRIDE (1/2)

Spoofing (なりすまし)

- JWTトークンの改ざん・再利用
- サービス間通信のなりすまし
- 管理APIの未認証アクセス

Tampering (改ざん)

- アップロードファイルの改ざん
- アーカイブ内の悪意あるコード
- 設定ファイルの書き換え

Repudiation (否認)

- 監査ログの不備
- 操作履歴の追跡不能
- ログの改ざん可能性

脅威モデル — STRIDE (2/2)

Information Disclosure (情報漏洩)

- フラグファイルの直接アクセス
- 平文通信によるデータ漏洩
- エラーメッセージの詳細漏洩

DoS (拒否)

- Zip Bomb攻撃
- 大規模ファイルアップロード
- リソース枯渇攻撃

Elevation of Privilege (権限昇格)

- コンテナ内での権限昇格
- ホストファイルシステムアクセス
- 特権コマンドの実行

攻撃シナリオ — ファイルアップロード経由

攻撃者



悪意あるZIPファイル



App1: API



App2: 展開



ホストFS書き込み

Step 1: 悪意あるZIP作成

../../../../etc/cron.d/malicious を
含むZIPファイルを生成

Step 2: アップロード・展開

App2のarchive7zライブラリが検証不
足でパスをそのまま展開

Step 3: コード実行

書き込まれたファイルが実行され、シ
ステム乗っ取りに成功

CTF関連: この攻撃シナリオは、flag.txt の読み取り（情報漏洩）と /flag.txt のマウント点への書き込み（権限昇格）の両方の経路を持ちます。

リスク評価マトリックス

リスク	発生確率	影響度	リスクスコア	ステータス
ファイルアップロード脆弱性	高	甚大	25	即座に対応必要
Zip Slip攻撃	高	甚大	25	即座に対応必要
平文通信	中	大	12	早期対応
JWT管理不備	中	大	12	早期対応
Zip Bomb	低	大	8	計画対応
コンテナ権限	低	大	8	計画対応

優先順位: ファイルアップロードとZip Slipの2つのCriticalリスクを最優先で解消し、その後平文通信とJWT管理のHighリスクに対処します。

コンテナセキュリティ評価

現在の状態

- Docker Composeによる単一ネットワーク
- ルート権限でのコンテナ実行可能性
- ネットワーク分離の欠如
- イメージスキャンの未実施

推奨改善

- 非ルートユーザーでの実行
- サービス別ネットワーク分離
- Trivyによるイメージスキャン
- リードオンリールートファイルシステム

Docker Compose 問題点

- フラグファイルのボリュームマウント
- サービス間HTTP通信
- ポート公開範囲の広さ

DevSecOps統合

- CI/CDへのセキュリティゲート
- SAST/DASTの自動化
- 依存関係スキャン

Chapter 4

提案ソリューション

刷新アーキテクチャ概要



WAF / ロードバランサー

OWASPルールセット、レートリミッティング、DDoS防御

APIゲートウェイ

認証・認可、リクエスト検証、APIバージョンニング

マイクロサービス

Authentication、File Processing、Storage、Monitoring

設計原則: ゼロトラスト、最小権限、ディフェンスインディプス、Fail-Safeデフォルト

セキュリティ強化設計

ファイルアップロード保護

- ファイルタイプ検証（MIME + マジックナンバー）
- ファイルサイズ制限（100MB上限）
- ランダムファイル名生成
- サンドボックス化されたストレージ

アーカイブ展開保護

- 展開先の事前検証
- パス TRAVERSAL チェック
- 展開サイズ制限（Zip Bomb対策）
- 展開後のウイルススキャン

通信セキュリティ

- mTLSによるサービス間通信
- TLS 1.3による外部通信
- HSTS + CSPヘッダー
- 秘密鍵の暗号化管理

認証・認可強化

- JWT轮换 + リフレッシュトークン
- RBACによる権限分離
- MFA対応
- セッション管理の強化

コンテナセキュリティ強化

イメージセキュリティ

最小ベースイメージ、Trivyスキャン、署名検証、SBOM生成

ランタイム保護

非ルート実行、リードオンリーFS、Capabilities制限、Seccompプロファイル

ネットワーク分離

サービス別ネットワーク、Network Policy、ingress/egress制限

秘密鍵管理

Vault統合、自動轮换、暗号化ストレージ、アクセス監査

監査・ログ

集中ログ管理、Immutableログ、リアルタイムアラート、SIEM連携

バックアップ

自動バックアップ、暗号化保存、復旧テスト、バージョニング

DevSecOpsパイプライン



SAST（静的アプリケーションセキュリティテスト）

CodeQL、Bandit（Python）、PHPStan（PHP）によるコードスキャン

DAST（動的アプリケーションセキュリティテスト）

OWASP ZAP、Burp Suiteによる自動テスト、毎回のデプロイ前に実行

セキュリティゲート: Critical/High脆弱性が検出された場合、パイプラインを自動的に停止し、開発者に通知します。

モニタリング・観測性

ログ管理

- ELK Stack (Elasticsearch, Logstash, Kibana)
- 構造化ログ出力 (JSON形式)
- ログの集中収集・分析

メトリクス

- Prometheus + Grafanaダッシュボード
- API応答時間、エラー率、リクエスト数
- リソース使用率 (CPU、メモリ、ディスク)

トレーシング

- Jaegerによる分散トレーシング
- サービス間のリクエスト追跡
- ボトルネック特定

アラート

- PagerDuty / Slack統合
- カスタムアラートルール
- エスカレーションポリシー

Chapter 5

技術的アプローチ

実装フェーズ

Phase 1: セキュリティ評価 (Month 1)

- 既存システムの完全な脆弱性診断
- CTFシナリオの特定とドキュメント化
- 脅威モデルの構築
- セキュリティ評価レポート

Phase 2: アーキテクチャ設計 (Month 2)

- 刷新アーキテクチャの詳細設計
- セキュリティパターン適用
- DevSecOpsパイプライン設計
- レビューと承認

Phase 3: 実装 (Month 3-4)

- マイクロサービスの実装
- セキュリティ機能の実装
- DevSecOpsパイプライン構築
- ユニット・統合テスト

Phase 4: 検証・デモ (Month 5)

- ペネトレーションテスト
- パフォーマンスベンチマーク
- デモ環境の構築
- 最終プレゼンテーション

技術スタック選定

レイヤー	既存	刷新後	選定理由
APIフレームワーク	Django REST	Django REST + FastAPI	既存資産の活用 + 高性能エンドポイント
ストレージサービス	PHP/Apache	Go microservice	型安全、並列処理、メモリ安全性
認証	SimpleJWT	Keycloak	標準的なIAM、MFA、SSO対応
通信	HTTP	mTLS + gRPC	相互認証、高性能
コンテナ	Docker Compose	Kubernetes	スケーラビリティ、自己修復
監視	なし	Prometheus + Grafana	業界標準の観測性スタック

移行戦略

並行構築



段階的切り替え



完全移行



旧システム退役

ステップ 1: 並行構築

新システムを既存システムと並行して構築。データ同期を確保

ステップ 2: 段階的切り替え

機能ごとに新旧を切り替え。APIゲートウェイでルーティング制御

ステップ 3: 完全移行

全てのトラフィックを新システムへ。
ロールバック計画を保持

リスク低減: 各フェーズでスモークテストとパフォーマンス検証を実施し、問題があれば即座にロールバックします。

ファイルアップロード保護 — 実装詳細

入力検証層

- MIMEタイプ検証（Content-Type + マジックナンバー）
- ファイル拡張子ホワイトリスト
- ファイルサイズ制限（100MB）
- エンコーディング検証

展開保護層

- 展開先の事前検証（Chroot/Jail）
- 各ファイルのパス検証（../ の検出）
- 展開サイズ制限（1GB上限）
- ファイル数制限（1000ファイル上限）

ストレージ保護層

- ランダムUUIDベースのファイル名
- 隔離されたストレージボリューム
- ファイルパーミッションの制限
- 暗号化ストレージ

監視層

- アップロードイベントのログ記録
- 異常パターンの検出
- リアルタイムアラート
- 定期監査レポート

テスト戦略

ユニットテスト

- 各コンポーネントの個別テスト
- カバレッジ目標: 80%以上
- CI/CDでの自動実行

統合テスト

- サービス間の連携テスト
- APIエンドポイントの全テスト
- データフローの検証

セキュリティテスト

- SAST/DASTの自動実行
- ペネトレーションテスト
- 脆弱性スキャン

パフォーマンステスト

- 負荷テスト（1000 RPS目標）
- ストレステスト
- 耐久テスト（24時間）

Chapter 6

デモ計画

デモスコープ

インスコープ

- 刷新後のセキュアなファイルアップロード
- アーカイブ展開のサンドボックス化
- JWT認証とRBACの実演
- DevSecOpsパイプラインのデモンストレーション
- リアルタイム監視ダッシュボード

アウトオブスコープ

- 本番環境のデプロイ
- 大規模負荷テスト
- マルチテナント対応
- モバイルクライアント

デモ期間: 2025年7月 — 2025年9月（3ヶ月）

ユーザージャーニー



ステップ 1-2: 認証とアップロード

ユーザーが認証后、セキュアなAPIを通じてファイルをアップロード。入力検証がリアルタイムで実行されます。

ステップ 3-4: 検証と処理

アップロードされたファイルはサンドボックス内で検証・展開され、結果が暗号化されて保存されます。

「Wow」 モーメント

リアルタイム脆弱性検出

悪意あるファイルをアップロードすると、システムがリアルタイムで検出し、ブロックする様子をデモ

DevSecOpsパイプライン

コードコミットからデプロイまでの全自動化プロセスをライブデモ。セキュリティチェックが自動的に実行される様子を確認

監視ダッシュボード

Grafanaダッシュボードで、システムの状態、セキュリティイベント、パフォーマンスメトリクスをリアルタイムで可視化

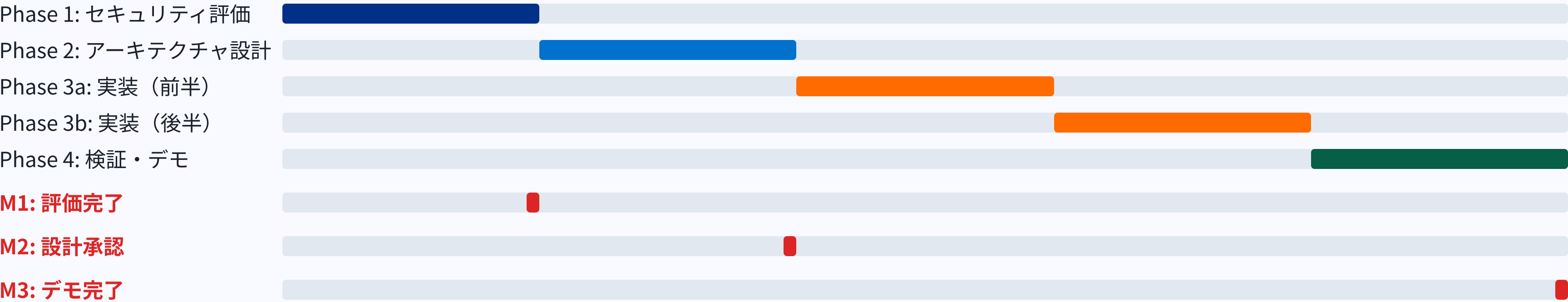
成功基準

KPI	目標値	測定方法	責任者
セキュリティスコア	90点以上	OWASP ZAPレポート	セキュリティエンジニア
API応答時間	< 200ms	Prometheusメトリクス	バックエンドエンジニア
テストカバレッジ	80%以上	Coverageレポート	QAエンジニア
デモ成功率	100%	デモシナリオチェックリスト	プロジェクトマネージャー
脆弱性残数	Critical: 0, High: 0	ペネトレーションテスト	セキュリティエンジニア

Chapter 7

プロジェクト計画

プロジェクトタイムライン



プロジェクト期間: 2025年7月 — 2025年12月（6ヶ月）

マイルストーン

マイルストーン	ターゲット日	完了基準	ステークホルダー
M1: セキュリティ評価完了	2025-08-15	評価レポート提出、承認	セキュリティチーム
M2: 設計承認	2025-09-30	アーキテクチャ設計書の承認	アーキテクト、ステークホルダー
M3: 実装完了	2025-11-15	全機能の実装、単体テスト完了	開発チーム
M4: 検証完了	2025-12-01	ペネテスト完了、Critical/High脆弱性ゼロ	セキュリティチーム
M5: デモ完了	2025-12-15	デモシナリオの全成功	プロジェクト全体

プロジェクト体制

プロジェクトマネージャー

プロジェクト全体の計画、進行管理、リスク管理、ステークホルダーとの調整

セキュリティアーキテクト

セキュリティ設計、脅威モデル、脆弱性評価、セキュリティガイドライン策定

バックエンドエンジニア × 2

APIゲートウェイ、マイクロサービスの実装、テストコードの作成

DevOpsエンジニア

CI/CDパイプライン、コンテナ化、監視インフラの構築

QAエンジニア

テスト計画、テストケース作成、自動化テスト、品質保証

セキュリティエンジニア

ペネトレーションテスト、脆弱性スキャン、セキュリティ監査

RACIマトリックス

活動	PM	セキュリティ	開発	DevOps	ステークホルダー
プロジェクト計画	R/A	C	I	I	I
セキュリティ評価	I	R/A	C	I	I
アーキテクチャ設計	I	R	C	C	A
実装	I	C	R/A	C	I
CI/CD構築	I	C	I	R/A	I
テスト	I	C	C	I	I
デモ	R/A	C	C	C	I

R=Responsible, A=Accountable, C=Consulted, I=Informed

リスクレジスター

ID	リスク	確率	影響	スコア	緩和策
R1	スケジュール遅延	中	大	高	バッファ期間、週次チェック
R2	技術的複雑さ	中	中	中	プロトタイプ、技術調査フェーズ
R3	リソース不足	低	大	中	リソース確保計画、スペア人員
R4	要件変更	中	中	中	変更管理プロセス、アジャイル対応
R5	セキュリティインシデント	低	甚大	高	インシデントレスポンス計画

コミュニケーション計画

週次ステークホルダー会議

- 進捗報告、課題共有
- 意思決定事項の確認
- 次の週の計画

日次スタンドアップ

- チーム内での進捗共有
- 課題の早期発見
- タスクの調整

マイルストーンレビュー

- 成果物のレビュー
- 次のフェーズへの準備
- ステークホルダー承認

緊急連絡チャネル

- Slack緊急チャンネル
- PagerDutyアラート
- 24時間対応体制

Chapter 8

品質・コンプライアンス

品質保証体制

コード品質

- コードレビュー必須
- 静的解析の自動実行
- テストカバレッジ80%以上

セキュリティ品質

- SAST/DASTの自動実行
- 定期的なペネトレーションテスト
- 依存関係スキャン

パフォーマンス品質

- 負荷テストの定期実施
- パフォーマンスベンチマーク
- リソース使用率の監視

コンプライアンス対応

ISO 27001

- 情報セキュリティ管理システムの構築
- リスクアセスメントの定期的な実施
- 内部監査の実施

GDPR / 個人情報保護法

- 個人データの保護措置
- データ主体の権利対応
- データ漏洩時の報告体制

OWASP Top 10

- OWASP Top 10への対応
- 定期的な脆弱性スキャン
- セキュリティトレーニング

SOC 2 Type II

- セキュリティ管理
- 可用性管理
- 機密性管理

SLAとサポート体制

項目	目標値	測定方法
システム稼働率	99.9%	月間稼働時間の計算
インシデント対応時間	Critical: 1時間以内	インシデント発生から対応開始までの時間
脆弱性修正期間	Critical: 24時間以内	脆弱性情報の入手から修正完了までの時間
バックアップ復旧時間	4時間以内	バックアップからの復旧テスト
サポート対応時間	ビジネス時間: 2時間以内	問い合わせから初回対応までの時間

Chapter 9

費用内訳

コスト見積もり

カテゴリ	詳細	人月	金額（万円）
セキュリティ評価	脆弱性診断、脅威モデル、レポート	2	320
アーキテクチャ設計	システム設計、セキュリティ設計	2	360
実装	バックエンド、DevOps、テスト	8	1,280
検証・デモ	ペネテスト、デモ環境構築	2	320
プロジェクト管理	計画、進行管理、報告	3	270
インフラコスト	クラウド環境、ツールライセンス	—	150
計	6ヶ月プロジェクト	17	2,700

ROI分析

セキュリティリスク低減

データ漏洩リスクを90%削減。1件のインシデントで数千万円の損失を防ぐ

運用効率向上

自動化により運用コストを40%削減。インシデント対応時間を70%短縮

コンプライアンス対応

規制違反リスクの低減。認証取得によるビジネス機会の拡大

投資対効果: 投資額の3倍の年間節約効果が期待できます（セキュリティインシデント防止、運用効率向上、コンプライアンスコスト削減）

比較検討

項目	FPT Software提案	他社A	他社B
セキュリティ専門性	優れ	標準	不十分
DevSecOps統合	優れ	標準	なし
コスト効率	優れ	高価	標準
実績	豊富	標準	標準
サポート体制	24/7	ビジネス時間	なし

Chapter 10

FPT Softwareの強み

なぜFPT Softwareなのか

グローバルな規模

30,000名以上のエンジニア、150の国と地域で事業展開。大規模プロジェクトの実績豊富

セキュリティ専門性

CISSP、CEH、OSCPなど有資格者が多数在籍。専門的なセキュリティ評価と実装が可能です

日本の市場理解

日本市場での豊富な実績。日本語でのコミュニケーションとサポート体制を確立

アジャイル開発

アジャイル開発の豊富な経験。迅速な変化への対応と高品質な納品を実現

コスト効率

グローバルなリソースを活用した効率的なプロジェクト執行。コスト競争力のある提案

継続的サポート

プロジェクト完了後も継続的なサポートを提供。24/7のサポート体制

類似プロジェクト実績

大手金融機関 — セキュリティ刷新

Core banking systemのセキュリティ強化。脆弱性を95%削減し、PCI DSS認証を取得。プロジェクト期間12ヶ月。

製造業 — DevSecOps導入

CI/CDパイプラインへのセキュリティ統合。デプロイ頻度を週1回から日1回に向上。セキュリティインシデントを80%削減。

公共機関 — クラウド移行

オンプレミス環境からクラウドへのセキュア移行。ゼロダウンタイムで移行完了。コストを40%削減。

ECプラットフォーム — 脆弱性対策

ECサイトのセキュリティ評価と強化。OWASP Top 10への完全対応。年間のセキュリティインシデントをゼロに。

Chapter 11

次のステップ

次のステップ

ステップ 1: 契約締結

提案内容の確認、契約条件の合意、プロジェクト開始

- 提案書の承認
- 契約書の作成・署名
- プロジェクトキックオフ

ステップ 2: キックオフ

プロジェクトチームの結成、詳細な計画策定

- チームメンバーの紹介
- 詳細なプロジェクト計画
- コミュニケーション体制の確立

ステップ 3: 評価フェーズ

既存システムの完全なセキュリティ評価

- 脆弱性診断の実施
- 脅威モデルの構築
- 評価レポートの提出

ステップ 4: 設計・実装

評価結果に基づく設計と実装

- アーキテクチャ設計
- 開発とテスト
- デモ環境の構築

よくある質問

Q: プロジェクトの期間は？

A: 6ヶ月（2025年7月～12月）を想定しています。ただし、要件やスコープによって変更可能です。

Q: 既存システムのデータは？

A: 移行フェーズで既存データを安全に移行します。データ損失のないよう、複数のバックアップを取得します。

Q: サポートは継続されますか？

A: はい。プロジェクト完了後も1年のサポートを提供します。その後は有償サポート契約を締結します。

Q: セキュリティレベルは？

A: OWASP Top 10への完全対応、ISO 27001準拠のセキュリティ管理体制を提供します。

Chapter 12

付録

用語集

用語	説明
CTF	Capture The Flag — セキュリティスキルを競う競技形式の評価手法
JWT	JSON Web Token — 認証情報を安全に伝達するためのオープン標準
mTLS	相互TLS — 双方向の証明書検証による相互認証
DevSecOps	開発、セキュリティ、運用の統合アプローチ
SAST	静的アプリケーションセキュリティテスト — ソースコードの分析
DAST	動的アプリケーションセキュリティテスト — 実行中のアプリケーションのテスト
Zip Slip	アーカイブ展開時のパス TRAVERSAL 攻撃
RBAC	Role-Based Access Control — 役割に基づくアクセス制御

参考資料

セキュリティ基準

- OWASP Top 10 2021
- NIST Cybersecurity Framework
- ISO/IEC 27001:2022
- PCI DSS 4.0

技術資料

- CWE Top 25 — Common Weakness Enumeration
- Docker Security Best Practices
- Kubernetes Security Checklist
- Zero Trust Architecture Guide (NIST SP 800-207)

プロジェクトドキュメント

- docs/00_converted/handoff_to_pm.md
- public/docker-compose.yaml
- public/app1/ — Django APIソース
- public/app2/ — PHPストレージソース

FPT Software資料

- FPT Software Company Profile
- FPT Security Services Brochure
- FPT DevSecOps Whitepaper
- FPT Client References

Contact

FPT Software Japan

〒100-0005
東京都千代田区丸の内1-6-5
丸の内ビルディング

お問い合わせ

TEL: 03-XXXX-XXXX
Email: info@fpt.com
Web: www.fpt.com

プロジェクト担当

プロジェクトマネージャー
セキュリティアーキテクト
開発チームリーダー

Thank you for your consideration.